

Hands-on 1: Spring Data JPA - Quick Example

Step 1: Create Spring Boot Project

Go to <https://start.spring.io> and choose:

- Project : Maven
- Language : Java
- Spring Boot : 2.x.x
- Group : com.cognizant
- Artifact : orm-learn

Add Dependencies:

- Spring Boot DevTools
- Spring Data JPA
- MySQL Driver

Generate the project and import it into Eclipse/IntelliJ.

Step 2: pom.xml

<dependencies>

```
<!-- Spring Boot Starter -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter</artifactId>
</dependency>

<!-- Spring Data JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL Driver -->
<dependency>
```

```
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<scope>runtime</scope>
</dependency>

<!-- Dev Tools -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>
```

Step 3: application.properties

Spring Logs

logging.level.org.springframework=info

logging.level.com.cognizant=debug

Hibernate Logs

logging.level.org.hibernate.SQL=trace

logging.level.org.hibernate.type.descriptor.sql=trace

Console Pattern

logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread %5p

%-25.25logger{25} %25M %4L %m%n

Database

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn

spring.datasource.username=root

spring.datasource.password=root

Hibernate

spring.jpa.hibernate.ddl-auto=validate

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect

Step 4: Create Database

create database ormlearn;

use ormlearn;

```
create table country(  
co_code varchar(2) primary key,  
co_name varchar(50)  
);
```

```
insert into country values('IN','India');
```

```
insert into country values('US','United States');
```

Step 5: Country.java

```
package com.cognizant.ormlearn.model;
```

```
import javax.persistence.Column;  
import javax.persistence.Entity;  
import javax.persistence.Id;  
import javax.persistence.Table;
```

```
@Entity
```

```
@Table(name="country")
```

```
public class Country {
```

```
    @Id
```

```
    @Column(name="co_code")
```

```
    private String code;
```

```
    @Column(name="co_name")
```

```
    private String name;
```

```
    public Country() {  
    }
```

```
    public Country(String code,String name){  
        this.code=code;  
        this.name=name;  
    }
```

```
    public String getCode(){  
        return code;  
    }
```

```
    public void setCode(String code){  
        this.code=code;  
    }
```

```

    public String getName(){
        return name;
    }

    public void setName(String name){
        this.name=name;
    }

    @Override
    public String toString(){
        return "Country [code="+code+", name="+name+"]";
    }
}

```

Step 6: CountryRepository.java

```

package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country,String>{

}

```

Step 7: CountryService.java

```

package com.cognizant.ormlearn.service;

import java.util.List;

import javax.transaction.Transactional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service

```

```

public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries(){

        return countryRepository.findAll();

    }

}

```

Step 8: OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.context.ApplicationContext;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService;

    public static void main(String[] args) {

        ApplicationContext context=
            SpringApplication.run(OrmLearnApplication.class,args);
    }
}

```

```

    LOGGER.info("Inside Main");

    countryService=context.getBean(CountryService.class);

    testGetAllCountries();

}

private static void testGetAllCountries(){

    LOGGER.info("Start");

    List<Country> countries=countryService.getAllCountries();

    LOGGER.debug("Countries={}",countries);

    LOGGER.info("End");

}

}

```

Output

Inside Main

Start

Countries=[Country [code=IN, name=India],
Country [code=US, name=United States]]

End

Hands-on 2: Difference between JPA, Hibernate and Spring Data JPA

Java Persistence API (JPA)

- JPA stands for Java Persistence API.
- It is a specification (JSR-338).

- It defines standards for Object Relational Mapping.
- JPA itself does not provide implementation.
- It only specifies interfaces and annotations.

Example:

```
@Entity
public class Employee{

}
```

Hibernate

- Hibernate is an ORM Framework.
- Hibernate implements JPA.
- It converts Java Objects into Database Tables.
- It handles SQL generation automatically.
- It supports HQL (Hibernate Query Language).

Example

```
Session session=factory.openSession();
```

```
Transaction tx=session.beginTransaction();
```

```
session.save(employee);
```

```
tx.commit();
```

```
session.close();
```

Spring Data JPA

- Spring Data JPA is built on top of Hibernate.
- It reduces boilerplate code.
- No need to create SessionFactory or Transaction manually.
- Repository interfaces provide CRUD operations automatically.

Example

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee,Integer>{
```

```
}
```

Saving an Employee

```
@Autowired
```

```
EmployeeRepository repository;
```

```
public void saveEmployee(Employee emp){
```

```
    repository.save(emp);
```

```
}
```

Comparison Table

JPA	Hibernate	Spring Data JPA
Specification	Framework	Abstraction over JPA
No implementation	Implements JPA	Uses Hibernate internally
Defines standards	ORM Tool	Repository-based ORM
Uses EntityManager	Uses SessionFactory	Uses JpaRepository
Less coding support	More coding	Minimum coding

Advantages of Spring Data JPA

- Less Boilerplate Code
- Automatic CRUD Operations
- Supports Pagination
- Supports Sorting
- Supports Query Methods
- Transaction Management
- Database Independent
- Easy Integration with Spring Boot

Output

Spring Boot Application Started Successfully

Retrieved Country List Successfully

Hands-on 3: Implement Services for Managing Country

Step 1: Create Country Entity

Country.java

```
package com.cognizant.ormlearn.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="country")
public class Country {

    @Id
    @Column(name="co_code")
    private String code;

    @Column(name="co_name")
    private String name;

    public Country() {

    }

    public Country(String code, String name) {
```

```

        this.code = code;

        this.name = name;
    }

    public String getCode() {

        return code;
    }

    public void setCode(String code) {

        this.code = code;
    }

    public String getName() {

        return name;
    }

    public void setName(String name) {

        this.name = name;
    }

    }

    @Override

    public String toString() {

        return "Country [code=" + code + ", name=" + name + "]";
    }

}

```

Step 2: Create Repository

CountryRepository.java

```

package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;

```

```
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

@Repository

public interface CountryRepository extends JpaRepository<Country,String>{

}
```

Step 3: Create Service

CountryService.java

```
package com.cognizant.ormlearn.service;

import java.util.List;

import javax.transaction.Transactional;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Service;

import com.cognizant.ormlearn.model.Country;

import com.cognizant.ormlearn.repository.CountryRepository;

@Service

public class CountryService {

    @Autowired

    private CountryRepository countryRepository;

    @Transactional

    public List<Country> getAllCountries(){

        return countryRepository.findAll();

    }

}
```

Step 4: Test

```
private static void testGetAllCountries(){  
    List<Country> countries=countryService.getAllCountries();  
    System.out.println(countries);  
}
```

Output

Country [code=IN, name=India]

Country [code=US, name=United States]

Hands-on 4: Find a Country Based on Country Code

Step 1: Create Exception

CountryNotFoundException.java

```
package com.cognizant.ormlearn.exception;  
  
public class CountryNotFoundException extends Exception{  
    public CountryNotFoundException(String message){  
        super(message);  
    }  
}
```

Step 2: Update CountryService.java

```
package com.cognizant.ormlearn.service;  
  
import java.util.Optional;
```

```

import javax.transaction.Transactional;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Service;

import com.cognizant.ormlearn.exception.CountryNotFoundException;

import com.cognizant.ormlearn.model.Country;

import com.cognizant.ormlearn.repository.CountryRepository;

@Service

public class CountryService {

    @Autowired

    private CountryRepository countryRepository;

    @Transactional

    public Country findCountryByCode(String code)

        throws CountryNotFoundException{

        Optional<Country> result=

            countryRepository.findById(code);

        if(!result.isPresent()){

            throw new CountryNotFoundException("Country Not Found");

        }

        return result.get();

    }

}

```

Step 3: Test

```

private static void testFindCountry(){

    try{

```

```
Country country=
    countryService.findCountryByCode("IN");

System.out.println(country);
}

catch(Exception e){

    System.out.println(e.getMessage());

}

}
```

Output

Country [code=IN, name=India]

Hands-on 5: Add a New Country

Step 1: Update CountryService.java

```
@Transactional

public void addCountry(Country country){

    countryRepository.save(country);

}
```

Step 2: Test

```
private static void testAddCountry(){

    Country country=

        new Country("JP","Japan");

    countryService.addCountry(country);

    System.out.println("Country Added Successfully");

}
```

```
}
```

Step 3: Verify

```
private static void verifyCountry(){  
    try{  
        Country country=  
            countryService.findCountryByCode("JP");  
        System.out.println(country);  
    }  
    catch(Exception e){  
        System.out.println(e.getMessage());  
    }  
}
```

Output

Country Added Successfully

Country [code=JP, name=Japan]